

LES PROTOCOLES MODBUS RTU ET TCP

Un RLI est un système de communication conçu pour relier plusieurs équipements industriels (capteurs, automates, actionneurs, etc.) au sein d'une zone géographique restreinte, souvent appelée « terrain ».

Ces réseaux sont également connus sous le nom de **bus de terrain** ou **réseaux de terrain**. Ils permettent l'échange de données entre divers dispositifs industriels, leur usage est limité à une zone géographique précise (atelier, usine, etc.) et plusieurs centaines de types de bus de terrain existent, offrant une large gamme de technologies adaptées aux besoins industriels spécifiques, les plus courants sont :

1. Modbus
2. Profibus
3. Interbus- S
4. ASI
5. LonWorks
6. Bus CAN

Modbus est l'un des protocoles de communication industriels les plus utilisés. Voici ses caractéristiques clés :

- Origine :
 - créé en 1979 par Modicon (rachetée en 1996 par Schneider Electric)
 - c'est un protocole non- propriétaire, donc libre d'utilisation et accessible publiquement.
- Usage :
 - il est principalement utilisé pour connecter des automates programmables
 - il intervient au niveau 7 du modèle OSI (niveau applicatif), gérant les interactions entre applications.
- Structure :
 - basé sur une architecture hiérarchique : un client unique interagit avec plusieurs serveurs
 - le protocole est conçu pour être simple et fiable, ce qui en fait un choix populaire pour les réseaux industriels.
- Avantages :
 - sa spécification est publique, facilitant son implémentation
 - il est dans le domaine public, permettant une adoption sans coût de licence.

Modbus reste aujourd'hui une technologie de référence dans l'automatisation industrielle en raison de sa simplicité, de sa compatibilité et de sa large adoption.

Modbus est un protocole de communication largement utilisé dans les réseaux d'automates programmables (API). Il repose sur une architecture **maître/esclave** pour l'échange de trames.

Ce modèle permet à un dispositif **maître** de contrôler et de communiquer avec **plusieurs dispositifs esclaves**. Les différents modes d'utilisation du protocole Modbus sont les suivants :

1. Modbus RTU (Remote Terminal Unit) :

Fonctionne directement sur des liaisons série telles que **RS-422, RS-485** ou **TTY** (boucle de courant).

- Caractéristiques :
 - protocole compact et efficace pour les communications industrielles ;
 - débits et distances variables selon la technologie de liaison série utilisée.
- Avantages :
 - idéal pour les systèmes nécessitant des échanges fiables sur des câbles série.

2. Modbus TCP/IP (ou Modbus TCP) :

Utilise les protocoles TCP/IP via des réseaux **Ethernet**.

- Caractéristiques :
 - protocole plus moderne, adapté aux environnements utilisant Ethernet pour les communications ;
 - permet une intégration facile dans des systèmes industriels connectés.
- Avantages :
 - offre des débits plus élevés ;
 - compatible avec les réseaux informatiques standards, facilitant l'interopérabilité.

3. Modbus Plus (ou Modbus+) :

Utilise un réseau à **passage de jetons** fonctionnant à une vitesse de **1 Mb/s**.

- Caractéristiques :
 - capable de transporter les trames Modbus ainsi que d'autres services spécifiques à ce réseau ;
 - conçu pour des environnements industriels nécessitant des échanges rapides et fiables.
- Avantages :
 - haute vitesse de transmission ;
 - gestion optimisée des trames grâce au mécanisme de jeton.

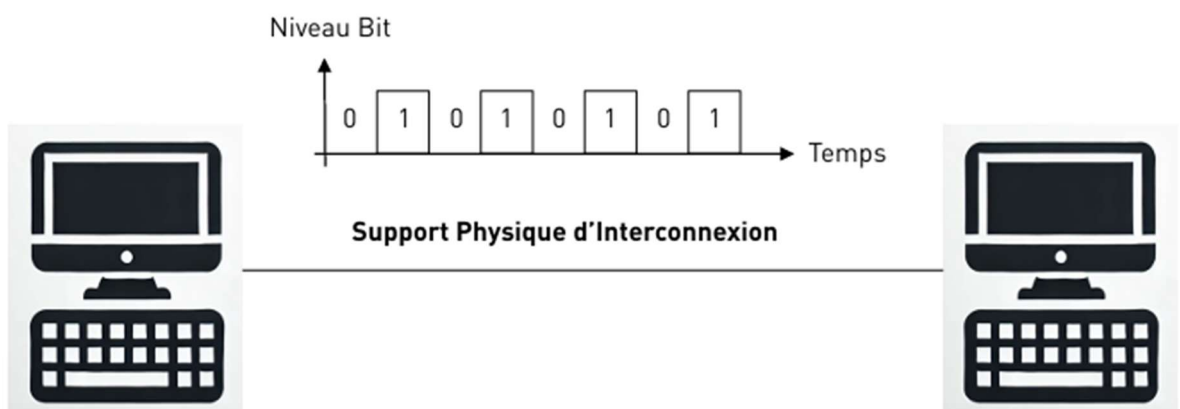
Modbus reste un protocole polyvalent, capable de s'adapter à différents types de réseaux et besoins industriels.

1. Modbus RTU est idéal pour les liaisons série traditionnelles.
2. Modbus TCP/IP répond aux exigences des systèmes connectés et modernes utilisant Ethernet.
3. Modbus Plus offre des performances élevées dans des environnements industriels complexes.

Grâce à sa simplicité et à sa flexibilité, Modbus demeure un standard incontournable dans l'automatisation industrielle.

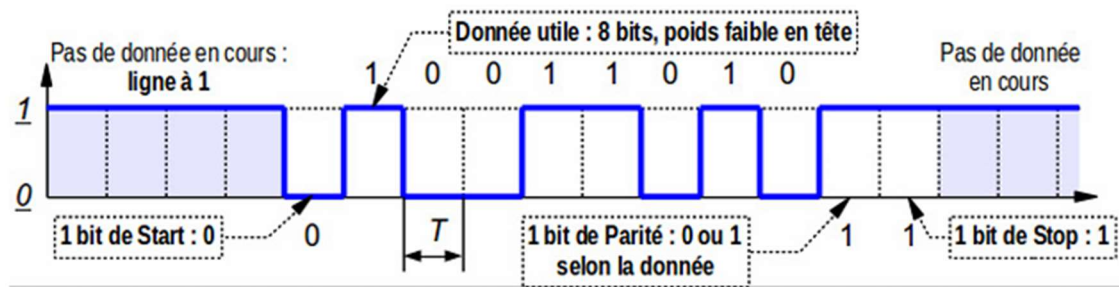
Selon des études récentes, Modbus TCP reste encore un protocole très utilisé.

Le protocole **Modbus RTU** repose sur une **liaison série** pour la transmission des données. Les bits sont envoyés séquentiellement au rythme d'une horloge, dont la période détermine la vitesse de transmission ou débit.



Les modes d'exploitation d'une liaison série déterminent la manière dont les données circulent entre les dispositifs connectés. Voici les trois principaux modes :

1. Mode Simplex :
 - la communication est unidirectionnelle : les données circulent uniquement dans une seule direction.
 - un dispositif agit comme émetteur et l'autre comme récepteur.
2. Mode Half Duplex :
 - la communication est bidirectionnelle, mais les données ne circulent que dans une direction à la fois ;
 - le même canal est utilisé pour l'émission et la réception, mais pas simultanément.
3. Mode Full Duplex :
 - la communication est bidirectionnelle simultanée, c'est-à-dire que les données peuvent être envoyées et reçues en même temps sur le même support physique, cela nécessite des mécanismes séparés pour gérer les deux flux de données.



Les trames en communication asynchrone se composent des éléments suivants :

- Bit de Start : une transition descendante signale au récepteur qu'il doit se synchroniser pour recevoir les données.
- Bits de Données : contiennent entre 7 et 8 bits représentant l'information transmise.
- Bit de Parité (optionnel) :
 - ce bit est généré à l'émission et vérifié à la réception pour garantir l'intégrité des données ;
 - avec une parité paire (even), le nombre total de bits (donnée + parité) doit être pair ;
 - avec une parité impaire (odd), le nombre total de bits (donnée + parité) doit être impair.
- Bit de Stop : Marque la fin de l'émission du caractère courant et permet de distinguer celui-ci du début (bit de Start) du caractère suivant. Sa durée peut être de 1, 1,5 ou 2 bits.

Il existe plusieurs normes pour les liaisons série, chacune adaptée à des besoins spécifiques :

- **RS232 (aussi appelée V24) :**
 - il s'agit d'une norme électrique définissant la transmission d'un signal sur un seul fil, référencé par rapport à la masse ;
 - elle est utilisée pour des **liaisons point à point**, où un seul émetteur et un seul récepteur sont connectés.
- **RS422 et RS485 :**
 - ces normes électriques définissent la transmission d'un signal sur un **support différentiel**, permettant une meilleure immunité au bruit ;
 - deux fils, transportant des signaux complémentaires, sont utilisés pour coder l'information ;
 - ces liaisons sont adaptées aux liaisons multipoint ou bus, où plusieurs dispositifs peuvent être connectés sur la même ligne de communication.

Ces normes permettent d'adapter les liaisons série aux exigences de distance, de débit et de robustesse dans divers environnements industriels.

Spécifications	RS 232	RS 422	RS 485
Type de communication	Unipolaire	Différentiel	Différentiel
Connexions électriques minimales	3 Fils Tx, Rx et Masse	5 Fils Paire Tx, Paire Rx et Masse	3 Fils Tx, Rx et Masse
Nombre de transmetteurs récepteurs alloués par ligne	1 Transmetteur 1 Récepteur	1 Transmetteur 31 Récepteurs	32 Transmetteurs 32 Récepteurs
Longueur maximum de câble	16,5 m	1320 m	1320 m
Débit maximum	64 Kbits/s	10 Mbits/s	10 Mbits/s

L'EIA-485, souvent appelée **RS-485**, est une norme définissant les caractéristiques électriques de la couche physique d'une interface numérique série. Elle est largement utilisée dans les réseaux industriels tels que **Modbus** et **Profibus**.

Caractéristiques principales :

1. Liaison multipoint :

- permet d'interconnecter plusieurs dispositifs sur le même bus ;
- jusqu'à 32 émetteurs et 32 récepteurs peuvent être connectés simultanément.

2. Architecture du bus :

- utilise un câblage différentiel pour la communication.
- configuration possible :
 - 2 fils pour un mode Half Duplex (communication alternée) ;
 - 4 fils pour un mode Full Duplex (communication simultanée dans les deux sens).

3. Portée :

- distance maximale atteignable d'environ 1 kilomètre en mode différentiel ;
- le mode différentiel améliore la tolérance aux perturbations électromagnétiques, garantissant une transmission fiable sur de longues distances.

4. Débit :

- supporte des débits élevés allant jusqu'à 10 Mbits/s ;
- cependant, la vitesse diminue à mesure que la distance augmente

En résumé grâce à sa robustesse, sa portée et sa flexibilité, le RS-485 est un choix privilégié pour les réseaux industriels nécessitant une communication fiable et efficace, même dans des environnements bruyants ou sur de longues distances.

Dans un système de communication basé sur une architecture maître- esclave, le fonctionnement suit des règles strictes pour garantir un échange structuré et sans conflit :

Règles de fonctionnement :

1. Communication initiée par le maître :

- le maître envoie une **requête** à un esclave spécifique, puis attend une réponse de celui- ci ;
- aucun esclave ne peut émettre de message de manière autonome

2. Identification des esclaves :

- chaque esclave est identifié par une **adresse unique** codée sur **8 bits** (soit 1 octet)
- cette adresse permet au maître de distinguer les différents esclaves connectés au réseau.

3. Restrictions de communication :

- les esclaves ne peuvent pas communiquer entre eux directement. Toute interaction doit passer par le maître.

4. Diffusion générale (Broadcast) :

- le maître peut envoyer un message à tous les esclaves présents sur le réseau simultanément ;
- cette diffusion générale utilise l'adresse spéciale 0 (adresse de broadcast).

Avantages de ce modèle

- Simplicité** : le maître contrôle entièrement les échanges, évitant les collisions de données.
- Prévisibilité** : le fonctionnement séquentiel garantit une communication ordonnée.
- Flexibilité** : la diffusion générale permet au maître de transmettre rapidement des instructions ou des informations globales.

Ce modèle est couramment utilisé dans les réseaux industriels comme Modbus, où une gestion centralisée des échanges est essentielle.

Les trames se déclinent en deux modes :

- Mode RTU (Remote Terminal Unit)** : Les données sont encodées sur 8 bits.
- Mode ASCII** : Les données sont représentées en ASCII, nécessitant deux caractères pour chaque octet.

Par exemple, l'octet 0x03 sera encodé sous la forme des caractères '0' et '3'.

Contenu :

- Un code fonction indiquant à l'esclave le type d'action demandée.
- Des données fournissant les informations complémentaires dont l'esclave a besoin pour exécuter la fonction.
- Un mot de contrôle pour permettre à l'esclave de vérifier l'intégrité des informations reçues.

Champ	Taille
N° station esclave	1 octet
Code fonction	1 octet
Information spécifique	n octets
Mot de contrôle	2 octets

La réponse

En cas de réussite :

- Le code fonction reste inchangé.
- Les données transmises contiennent le résultat de la requête.
- Le mot de contrôle garantit l'intégrité des données envoyées.

Champ	Taille
N° station esclave	1 octet
Code fonction	1 octet
Données transmises	n octets
Mot de contrôle	2 octets

En cas d'erreur :

- Le code fonction est modifié pour indiquer une réponse d'exception (MSB = 1).
- Les données contiennent un code d'exception permettant de connaître le type d'erreur.

Champ	Taille
N° station esclave	1 octet
Code fonction + bit d'erreur	1 octet
Code d'exception	1 octet
Mot de contrôle	2 octets

Codes d'Exception

- 01 : Fonction illégale (erreur sur le code fonction).
- 02 : Erreur sur l'adresse du registre ou du coil.
- 08 : Erreur de transmission (contrôle CRC ou Timing).

Le mot de contrôle d'une trame Modbus est un code de vérification d'erreur appelé contrôle de redondance cyclique sur 16 bits (CRC16).

- Le CRC (CyclicalRedundancy Check) est calculé par l'émetteur avant l'envoi de la trame.
- Le récepteur, à son tour, effectue un calcul de CRC sur la trame reçue et le compare avec le CRC transmis. Si les deux valeurs diffèrent, cela indique une erreur dans la transmission du message.

Le CRC utilisé dans le protocole Modbus repose sur un calcul basé sur une opération de **OU exclusif (XOR)**.

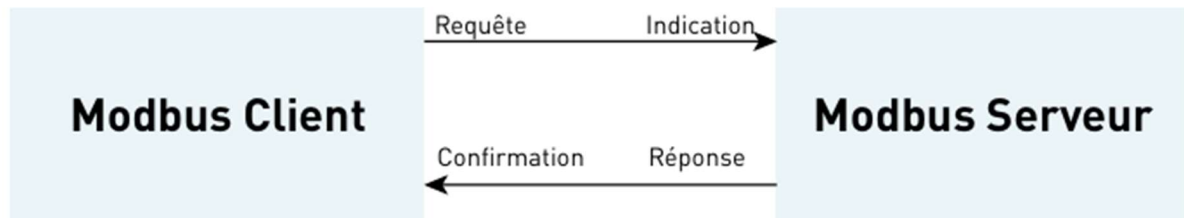
Le protocole Modbus propose 19 fonctions différentes, mais tous les équipements ne prennent pas en charge l'ensemble de ces codes de fonctions.

Code	Nature des fonctions MODBUS	TSX 37
H'01'	Lecture de n bits de sortie consécutifs	X
H'02'	Lecture de n bits de sortie consécutifs	X
H'03'	Lecture de n mots de sortie consécutifs	X
H'04'	Lecture de n mots consécutifs d'entrée	X
H'05'	Écriture de 1 bit de sortie	X
H'06'	Écriture de 1 mot de sortie	
H'07'	Lecture du statut d'exception	
H'08'	Accès aux compteurs de diagnostic	
H'09'	Téléchargement, télé déchargement et mode de marche	
H'0A'	Demande de CR de fonctionnement	
H'0B'	Lecture du compteur d'événements	X
H'0C'	Lecture des événements de connexion	X
H'0D'	Téléchargement, télé déchargement et mode de marche	
H'0E'	Demande de CR de fonctionnement	

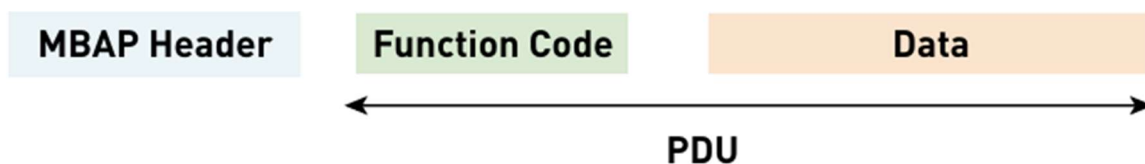
Code	Nature des fonctions MODBUS	TSX 37
H'0F'	Écriture de n bits de sortie	X
H'10'	Écriture de n mots de sortie	X
H'11'	Lecture d'identification	X
H'12'	Téléchargement, télé déchargement et mode de marche	
H'13'	Reset de l'esclave après erreur non recouvrée	

Modbus utilise une architecture Client/Serveur pour établir des connexions et échanger des données entre les équipements. Le Serveur Modbus écoute les requêtes sur le port TCP 502. Son fonctionnement peut être décrit ainsi :

- Le module Client Modbus crée une requête à partir des informations fournies par l'application.
- Le module Serveur Modbus reçoit ces requêtes et exécute les actions nécessaires, comme des lectures ou des écritures, afin d'y répondre.



Le protocole Modbus spécifie une unité de données de protocole (PDU), qui est indépendante des autres couches de communication. Lorsqu'il est encapsulé sur TCP/IP, un champ supplémentaire, appelé **MBAP Header**, est ajouté.



Le contenu du MBAP Header est le suivant :

Champs	Longueur	Description	Client	Serveur
Transaction Identifier	2 Octets	Identification d'une Transaction de Requête/ Réponse Modbus	Initialisé par le Client	Copié par le Serveur à partir de la demande reçue
Protocol Identifier	2 Octets	0 = Protocole Modbus	Initialisé par le Client	Copié par le Serveur à partir de la demande reçue
Longueur	2 Octets	Nombre d'Octets Suivant	Initialisé par le Client (Requête)	Initialisé par le Serveur (Réponse)
Unit Identifier	1 Octet	Identification d'un esclave distant connecté sur une ligne série ou sur d'autres bus	Initialisé par le Client	Copié par le Serveur à partir de la demande reçue

Remarque : Le champ "Unit Identifier" est utilisé pour le routage lorsqu'un périphérique situé sur un réseau série Modbus est accessible via une passerelle (Gateway). Dans ce cas, ce champ contient l'adresse esclave de l'appareil distant, tandis que l'adresse IP identifie la passerelle, et non l'esclave. Si le champ "Unit Identifier" n'est pas requis, il doit être défini sur la valeur 0xFF. Cependant, en pratique, ce champ est souvent fixé à 1 et est considéré comme l'adresse du serveur.

Le champ Function Code du PDU peut avoir les valeurs suivantes :

Le champ **Function Code** indique à l'esclave adressé quelle fonction doit être exécutée. Les fonctions suivantes sont prises en charge par **Modbus Poll** :

- 01 (0x01)** : Lecture des bobines (**Read Coils**)
- 02 (0x02)** : Lecture des entrées discrètes (**Read Discrete Inputs**)
- 03 (0x03)** : Lecture des registres de maintien (**Read Holding Registers**)
- 04 (0x04)** : Lecture des registres d'entrée (**Read Input Registers**)
- 05 (0x05)** : Écriture d'une seule bobine (**Write Single Coil**)
- 06 (0x06)** : Écriture d'un seul registre (**Write Single Register**)
- 08 (0x08)** : Diagnostics (**Diagnostics**, uniquement pour les lignes série)
- 11 (0x0B)** : Lecture du compteur d'événements de communication (**Get Comm Event Counter**, uniquement pour les lignes série)
- 15 (0x0F)** : Écriture de plusieurs bobines (**Write Multiple Coils**)
- 16 (0x10)** : Écriture de plusieurs registres (**Write Multiple Registers**)
- 17 (0x11)** : Rapport d'identifiant du serveur (**Report Server ID**, uniquement pour les lignes série)
- 22 (0x16)** : Écriture de masques dans les registres (**Mask Write Register**)
- 23 (0x17)** : Lecture/Écriture de plusieurs registres (**Read/Write Multiple Registers**)
- 43 / 14 (0x2B / 0x0E)** : Lecture de l'identification du dispositif (**Read Device Identification**)

Voici une capture de trames du protocole Modbus :

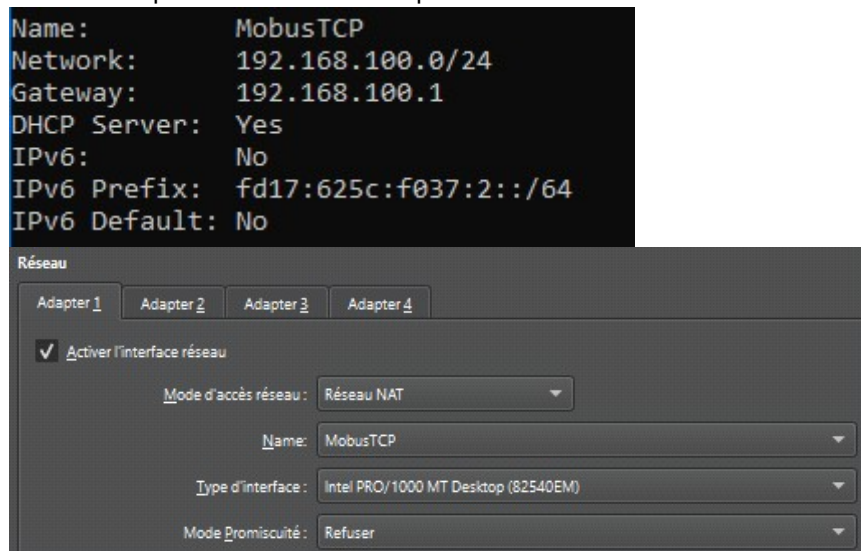
En résumé le protocole Modbus est très utilisé dans l'industrie :

- Modbus RTU : C'est l'ancêtre du protocole Modbus, utilisé sur des liaisons séries RS-485, de moins en moins utilisé dans l'industrie au profit du protocole....
- Modbus TCP : Ce protocole fonctionne en architecture Client/Serveur et le serveur écoute sur le port TCP 502

Exercice mise en place du protocole Modbus TCP

Voici un exemple d'exercice visant à mettre en place le protocole Modbus TCP en simulant un **serveur** et un **client Modbus TCP**. Pour cela, nous utiliserons 2 machines virtuelles, 1 sous **Linux Ubuntu Desktop** et 1 autre sous **Linux Ubuntu Serveur**. La machine virtuelle sous Linux Ubuntu Serveur simulera le **serveur Modbus** et la machine virtuelle sous Linux Ubuntu Desktop simulera le **client Modbus**.

Pour ce faire je vais procéder à l'installation de 2 machines virtuelles avec un réseau NAT virtuel réseau 192.168.100.0/24, il faut que ces machines puissent communiquer entre elles et puissent accéder à Internet.



Il faut ensuite vérifier que python3 est installé sur les 2 machines virtuelles.

```
dnvn@dnvn-VirtualBox:~$ python3
Python 3.10.12 (main, Jan 26 2026, 14:55:28) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
dnvn@dnvn-server:~$ python3
Python 3.12.3 (main, Jan 22 2026, 20:57:42) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> _
```

Nous utiliserons la librairie suivante pour réaliser la simulation du protocole Modbus TCP :<https://pymodbustcp.readthedocs.io/en/latest/>

Dans un premier temps il faut installer et activer un environnement virtuel python sur le **serveur** :

```
dnvn@dnvn-server:~$ python3 -m venv server-modbus
dnvn@dnvn-server:~$ source server-modbus/bin/activate
(server-modbus) dnvn@dnvn-server:~$ _
```

Ensuite installer et activer un environnement virtuel python également sur le client :

```
dnvn@dnvn-VirtualBox:~$ python3 -m venv server-modbus
dnvn@dnvn-VirtualBox:~$ source server-modbus/bin/activate
(server-modbus) dnvn@dnvn-VirtualBox:~$
```

Maintenant il faut installer les librairies pour le programme python aussi bien sur le serveur que sur le client :

```
(server-modbus) dnvn@dnvn-server:~$ pip install pyModbusTCP
Requirement already satisfied: pyModbusTCP in ./server-modbus/lib/python3.12/site-packages (0.3.0)
(server-modbus) dnvn@dnvn-server:~$ pip install DataBank
Requirement already satisfied: DataBank in ./server-modbus/lib/python3.12/site-packages (1.1.0)
Requirement already satisfied: sqlalchemy<=2.0.4 in ./server-modbus/lib/python3.12/site-packages (from DataBank) (2.0.46)
Requirement already satisfied: greenlet<=1 in ./server-modbus/lib/python3.12/site-packages (from sqlalchemy<=2.0.4->DataBank) (3.3.1)
Requirement already satisfied: typing-extensions<=4.6.0 in ./server-modbus/lib/python3.12/site-packages (from sqlalchemy<=2.0.4->DataBank) (4.15.0)
(server-modbus) dnvn@dnvn-server:~$ pip install uniform
Requirement already satisfied: uniform in ./server-modbus/lib/python3.12/site-packages (0.2.2)
Requirement already satisfied: typesystem in ./server-modbus/lib/python3.12/site-packages (from uniform) (0.4.1)
Requirement already satisfied: starlette in ./server-modbus/lib/python3.12/site-packages (from uniform) (0.52.1)
Requirement already satisfied: anyio<5,>=3.6.2 in ./server-modbus/lib/python3.12/site-packages (from starlette->uniform) (4.12.1)
Requirement already satisfied: typing-extensions<=4.10.0 in ./server-modbus/lib/python3.12/site-packages (from starlette->uniform) (4.15.0)
Requirement already satisfied: idna<=2.8 in ./server-modbus/lib/python3.12/site-packages (from anyio<5,>=3.6.2->starlette->uniform) (3.11)
(server-modbus) dnvn@dnvn-server:~$
```

```
(server-modbus) dnvn@dnvn-VirtualBox:~$ pip install pyModbusTCP
Collecting pyModbusTCP
  Downloading pyModbusTCP-0.3.0-py3-none-any.whl (24 kB)
Installing collected packages: pyModbusTCP
Successfully installed pyModbusTCP-0.3.0
(server-modbus) dnvn@dnvn-VirtualBox:~$ pip install DataBank
Collecting DataBank
  Using cached databank-0.8.0-py3-none-any.whl (7.7 kB)
Collecting sqlalchemy<3.0.0,>=2.0.4
  Using cached sqlalchemy-2.0.46-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (3.2 MB)
Collecting greenlet<=1
  Using cached greenlet-3.3.1-cp310-cp310-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (586 kB)
Collecting typing-extensions<=4.6.0
  Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Installing collected packages: typing-extensions, greenlet, sqlalchemy, DataBank
Successfully installed DataBank-0.8.0 greenlet-3.3.1 sqlalchemy-2.0.46 typing-extensions-4.15.0
(server-modbus) dnvn@dnvn-VirtualBox:~$ pip install uniform
Collecting uniform
  Using cached uniform-0.2.2-py3-none-any.whl (5.2 kB)
Collecting starlette
  Using cached starlette-0.52.1-py3-none-any.whl (74 kB)
Collecting typesystem
  Using cached typesystem-0.4.1-py3-none-any.whl (28 kB)
Requirement already satisfied: typing-extensions<=4.10.0 in ./server-modbus/lib/python3.10/site-packages (from starlette->uniform) (4.15.0)
Collecting anyio<5,>=3.6.2
  Using cached anyio-4.12.1-py3-none-any.whl (113 kB)
Collecting exceptiongroup<=1.0.2
  Using cached exceptiongroup-1.3.1-py3-none-any.whl (16 kB)
Collecting idna<=2.8
  Downloading idna-3.11-py3-none-any.whl (71 kB)
 71.0/71.0 KB 634.6 kB/s eta 0:00:00
Installing collected packages: typesystem, idna, exceptiongroup, anyio, starlette, uniform
Successfully installed anyio-4.12.1 exceptiongroup-1.3.1 idna-3.11 starlette-0.52.1 typesystem-0.4.1 uniform-0.2.2
(server-modbus) dnvn@dnvn-VirtualBox:~$
```

(server-modbus) indique que l'environnement virtuel Python est activé et donc utilise les librairies installées dans server-modbus, pyModbusTCP est disponible. Sans ça Python utiliserait le Python global du système, les librairies installées dans server-modbus ne seront PAS accessibles et mon script pourrait afficher une erreur.

Si **server.py** & **client.py** ne sont pas créés automatiquement, il faut le faire manuellement avec `sudo nano` :

```
GNU nano 7.2 server.py *
#!/usr/bin/env python3

#####
Modbus/TCP server
#####
Run this as root to listen on TCP privileged ports (<= 1024).

Add "--host 0.0.0.0" to listen on all available IPv4 addresses of the host.
$ sudo ./server.py --host 0.0.0.0
#####
import argparse
import logging

from pyModbusTCP.server import ModbusServer

# init logging
logging.basicConfig()
# parse args
parser = argparse.ArgumentParser()
parser.add_argument('-H', '--host', type=str, default='localhost', help='Host(default: localhost)')
parser.add_argument('-p', '--port', type=int, default='5020', help='TCP port(default: 5020)')
parser.add_argument('-d', '--debug', action='store_true', help='set debug mode')
args = parser.parse_args()
# logging setup
if args.debug:
    logging.getLogger('pyModbusTCP.server').setLevel(logging.DEBUG)
# start modbus server
server = ModbusServer(host=args.host, port=args.port, no_block=False)
server.start()
```

Le script **server.py** importe trois éléments importants.

Le module `argparse` sert à gérer les arguments passés en ligne de commande. Cela permet à l'utilisateur de préciser certains paramètres au moment du lancement du serveur.

Le module `logging` est utilisé pour afficher des messages d'information ou de débogage.

Enfin, la classe `ModbusServer`, provenant de la bibliothèque `pyModbusTCP`, est l'élément central : elle permet de **créer concrètement le serveur Modbus TCP**.

Une partie essentielle du script consiste à définir des arguments configurables.

Grâce à `argparse`, l'utilisateur peut préciser l'adresse IP d'écoute avec l'option `--host`, le port TCP avec l'option `--port`, ainsi qu'activer un mode débogage avec l'option `--debug`.

Par défaut, le serveur écoute sur l'adresse localhost et sur le port 502, qui est le port standard du protocole Modbus TCP. Une alternative plus simple pour un TP consiste à choisir un port supérieur à 1024, comme 5020, afin d'éviter d'avoir besoin des droits root.

Si l'option `--debug` est utilisée, le niveau de détail des messages affichés dans le terminal augmente, ce qui peut être utile pour analyser le fonctionnement du serveur.

Enfin, le script crée une instance du serveur Modbus en utilisant les paramètres fournis, puis appelle la méthode `start()`.

À partir de ce moment, le serveur se met en écoute sur le réseau et attend les connexions des clients. Le terminal reste alors bloqué, ce qui est normal : cela signifie que le service est en fonctionnement.

Il est important de noter que le port 502 est un port privilégié sous Linux. Cela signifie que pour l'utiliser, il faut exécuter le script avec les droits administrateur (par exemple avec `sudo`).

En résumé, ce script permet de configurer puis de lancer un serveur Modbus TCP personnalisable, qui servira de point d'échange de données avec le client Modbus installé sur l'autre machine virtuelle.

```
#!/usr/bin/env python3
"""Write 4 coils to True, wait 2s, write False and redo it."""

import time
from pyModbusTCP.client import ModbusClient

# init
c = ModbusClient(host='192.168.100.6', port=5020, auto_open=True)
bit = True

# main loop
while True:
    # write 4 bits in modbus address 0 to 3
    print('write bits')
    print('-----\n')
    for ad in range(4):
        is_ok = c.write_single_coil(ad, bit)
        if is_ok:
            print('coil %s: write to %s' % (ad, bit))
        else:
            print('coil %s: unable to write %s' % (ad, bit))
        time.sleep(0.5)
    print('')
    time.sleep(1)

    # read 4 bits in modbus address 0 to 3
    print('read bits')
    print('-----\n')
    bits = c.read_coils(0, 4)
    if bits:
        print('coils #0 to 3: %s' % bits)
    else:
        print('coils #0 to 3: unable to read')
    # toggle
    bit = not bit
    # sleep 2s before next polling
```

```
# sleep 2s before next polling
print('')
time.sleep(2)
```

^G Aide	^O Écrire	^W Chercher	^K Couper	^T Exécuter	^C Emplacement
^X Quitter	^R Lire fich.	^_ Remplacer	^U Coller	^J Justifier	^/ Aller ligne

Le script **client.py** est un programme Python qui joue le rôle de client Modbus TCP. Il se connecte au serveur Modbus (celui lancé avec **server.py**) et échange des données avec lui de manière répétée.

Au début du script, la ligne `#!/usr/bin/env python3` indique que le programme doit être exécuté avec Python 3. Ensuite, deux modules sont importés : le module `time`, utilisé pour gérer des temporisations. Le client est ensuite initialisé avec la classe `ModbusClient` provenant de la bibliothèque `pyModbusTCP`, puis `c = ModbusClient(host='xxx.xx.x.xxx', port=5020, auto_open=True)` permet d'établir une connexion avec le serveur Modbus TCP.

Cela signifie que le programme tente de se connecter au serveur situé à l'adresse IP xxx.xx.x.xxx sur le port 5020. Le paramètre `auto_open=True` permet au client d'ouvrir automatiquement la connexion si elle n'est pas déjà établie au moment d'envoyer une requête.

Une variable nommée `bit` est initialisée à la valeur `True`. Elle servira à écrire alternativement des valeurs vraies puis fausses dans les coils du serveur.

Le cœur du programme se trouve dans une boucle infinie (`while True`). À chaque cycle, le client effectue plusieurs actions :

Premièrement, il écrit quatre coils (sorties logiques) aux adresses 0 à 3 du serveur. Pour chaque adresse, il envoie la valeur contenue dans la variable `bit`. Si l'écriture réussit, un message de confirmation s'affiche dans le terminal ; sinon, un message d'erreur est affiché. Une courte pause de 0,5 seconde est insérée entre chaque écriture afin de ralentir les échanges et les rendre lisibles.

Deuxièmement, le client lit les quatre mêmes coils (adresses 0 à 3) en utilisant la fonction `read_coils(0, 4)`. Si la lecture réussit, les valeurs récupérées sont affichées. Sinon, un message indique que la lecture a échoué.

Ensuite, la variable `bit` est inversée grâce à l'instruction `bit = not bit`. Cela signifie que si la valeur était `True`, elle devient `False`, et inversement. Ainsi, à chaque cycle de la boucle, le client alterne entre l'écriture de valeurs vraies et fausses.

Enfin, le programme attend deux secondes avant de recommencer un nouveau cycle.

En résumé, ce script se connecte au serveur Modbus TCP, écrit quatre sorties logiques, les relit pour vérifier leur état, inverse leur valeur, puis recommence indéfiniment. Il permet ainsi de tester le bon fonctionnement de la communication Modbus entre la machine cliente (Ubuntu Desktop) et la machine serveur (Ubuntu Server).

Maintenant, si l'on lance dans un premier temps le programme server.py, puis dans un second temps le programme client.py avec :

```
sudo python3 server.py
```

```
sudo python3 client.py
```

La commande sudo python3 server.py échoue, car sudo ignore notre venv (virtualenvironment) :

```
(server-modbus) dnv@dnvserver:~$ sudo python3 server.py
[sudo] password for dnv:
Traceback (most recent call last):
  File "/home/dnv/server.py", line 14, in <module>
    from pyModbusTCP.server import ModbusServer
ModuleNotFoundError: No module named 'pyModbusTCP'
(server-modbus) dnv@dnvserver:~$ pip install pyModbusTCP
Requirement already satisfied: pyModbusTCP in ./server-modbus/lib/python3.12/site-packages (0.3.0)
(server-modbus) dnv@dnvserver:~$ python server.py --port 5020
```

File "/home/dnv/server.py", line 14 in <module>frompyModbusTCP.server import ModbusServerModuleNotFounError: No module named 'pyModbusTCP'

Un venv est un environnement Python isolé, installé dans un dossier du projet. C'est dans cet environnement que Python et les modules comme pyModbusTCP sont installés.

Le venv permet :

- D'avoir des versions spécifiques de modules
- D'isoler un projet des autres
- D'éviter les conflits de dépendances
- De ne pas modifier le Python système

Par exemple :

- Projet A → pyModbusTCP 0.3.0
- Projet B → autre version
- Le système Linux → reste propre

Il faut donc activer le venv(ce qui est censé déjà être le cas) et lancer le script avec :source server-modbus/bin/activate&&python server.py --port 5020

On observe les messages liés à notre script client "**Unable To Write**",c'est parce que dans le script server.py par défaut host = localhost (127.0.0.1). Ça fait que le serveur n'écoute que sur l'interface locale de la VM serveur.

Quand on met explicitement python server.py --host 0.0.0.0 --port 5020

0.0.0.0 = toutes les interfaces réseau de la VM, on autorise donc le serveur à donner l'accès au client via son IP réseau.

Si on veut être plus sélectif au niveau des connexions autorisées on peut écouter seulement sur une IP précise

Au lieu de 0.0.0.0, il faudrait mettre l'IP réseau spécifique de la VM cliente :

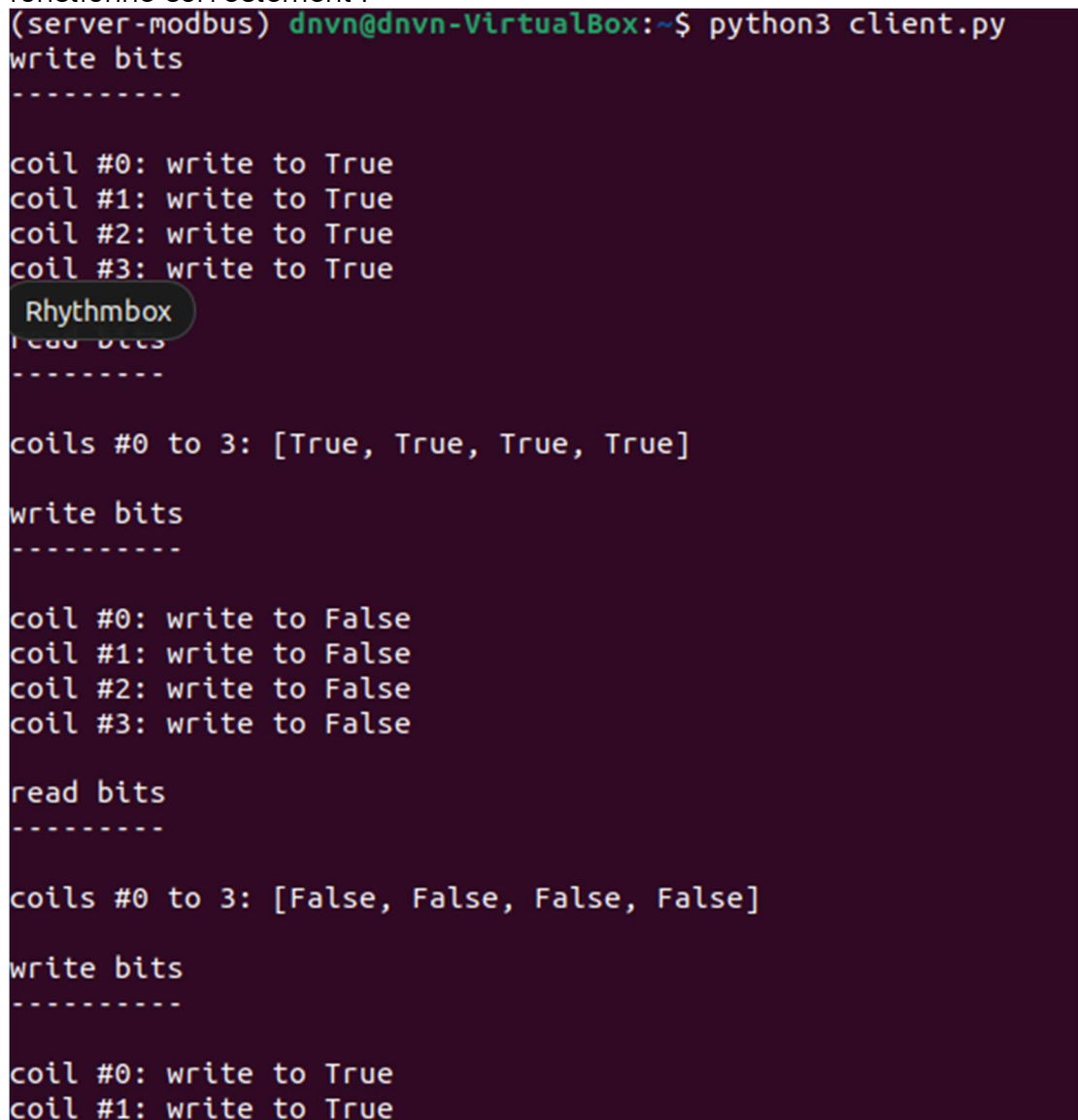
```
python server.py --host «IP_VM_Client» --port 5020
```

Ainsi le serveur écoutera uniquement sur cette interface précise.

Sinon on peut Filtrer avec le pare-feu avecufw :

sudo firewall rule add from 192.168.1.20 to any port 5020 autoriser uniquement une IP précise et pour bloquer les autres : sudo firewall rule add deny 5020. Résultat, seule l'IP 192.168.1.20 pourra se connecter.

Lors du lancement de script client le terminal devrait montrer que le programme fonctionne correctement :



```
(server-modbus) dnv@dnv-VirtualBox:~$ python3 client.py
write bits
-----
coil #0: write to True
coil #1: write to True
coil #2: write to True
coil #3: write to True
Rhythmbox
read bits
-----
coils #0 to 3: [True, True, True, True]

write bits
-----
coil #0: write to False
coil #1: write to False
coil #2: write to False
coil #3: write to False

read bits
-----
coils #0 to 3: [False, False, False, False]

write bits
-----
coil #0: write to True
coil #1: write to True
```

Le programme fonctionne correctement, il envoie bien 4 trames **Modbus** en **True**, il lit les informations qui correspondent à **[True, True, True, True]**, il attend 2 secondes, il envoie bien 4 trames **Modbus** en **False**, il lit les informations qui correspondent à **[False, False, False, False]** et il répète jusqu'à l'arrêt du programme.

Maintenant que le protocole Modbus TCP a été mis en place, nous allons analyser les échanges des trames Modbus TCP entre le serveur et le client. Et pour cela je vais utiliser Wireshark `sudo apt install wireshark && sudo wireshark`.

Il faut lancer dans un premier temps le programme **server.py** puis lancer la capture de trames Wireshark sur le **client (Ubuntu Desktop)** et ensuite lancer le programme **client.py**, puis arrêter la capture de trames.

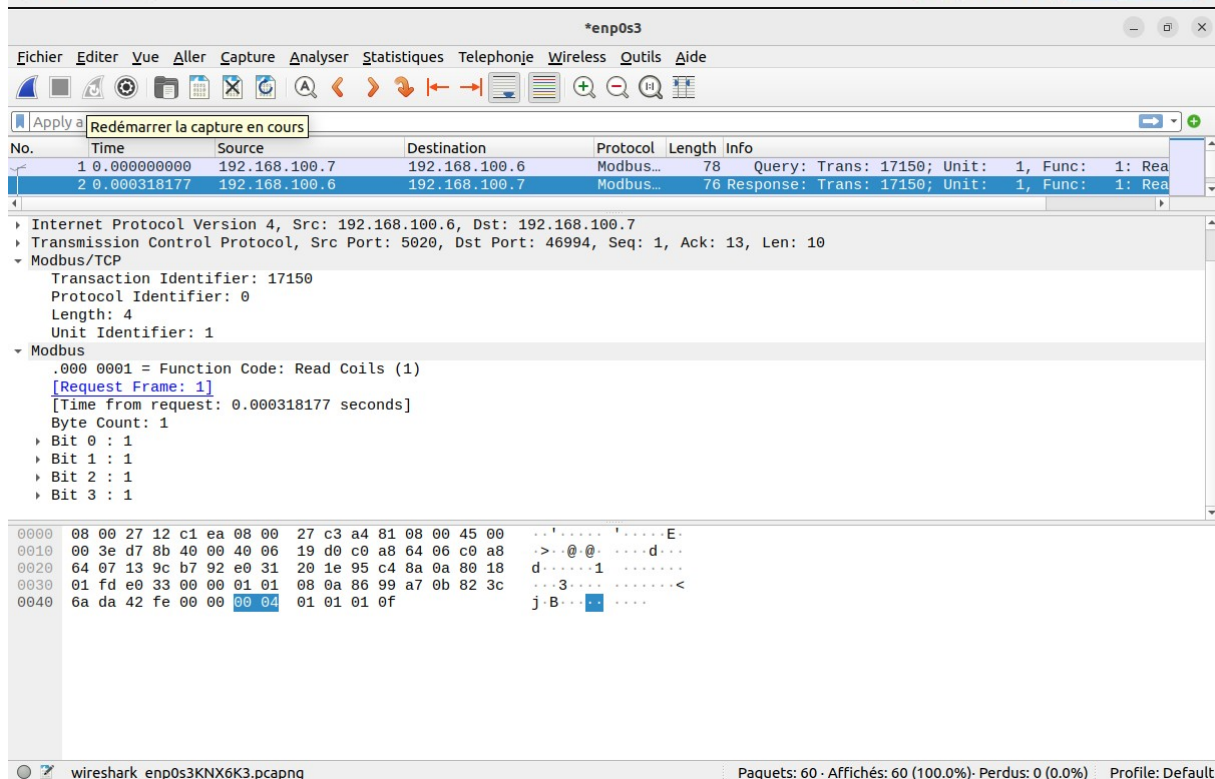
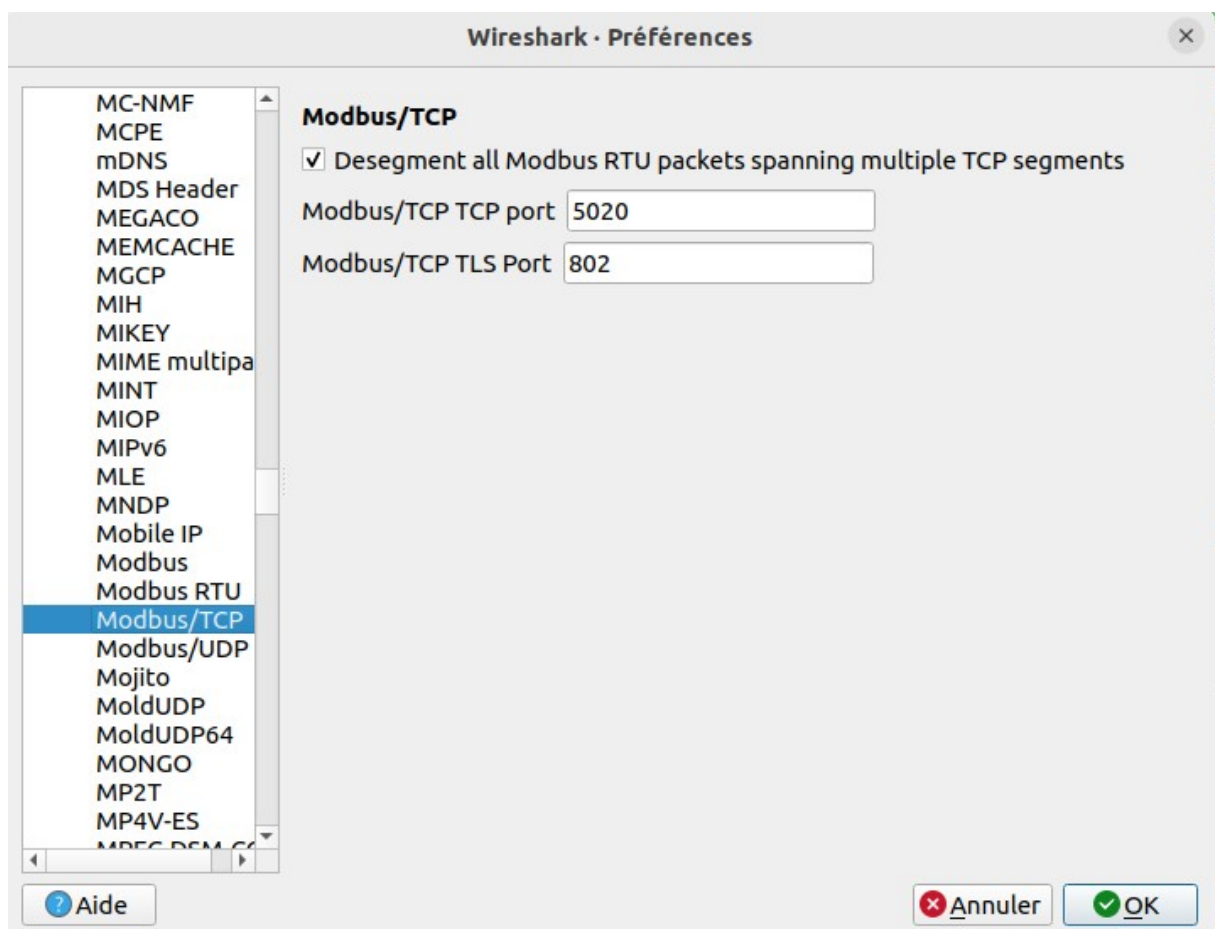
Voici la capture de trames Modbus TCP des échanges entre le client et le serveur sur un cycle complet du programme, envoi et réception de [True, True, True, True] puis de [False, False, False, False] :

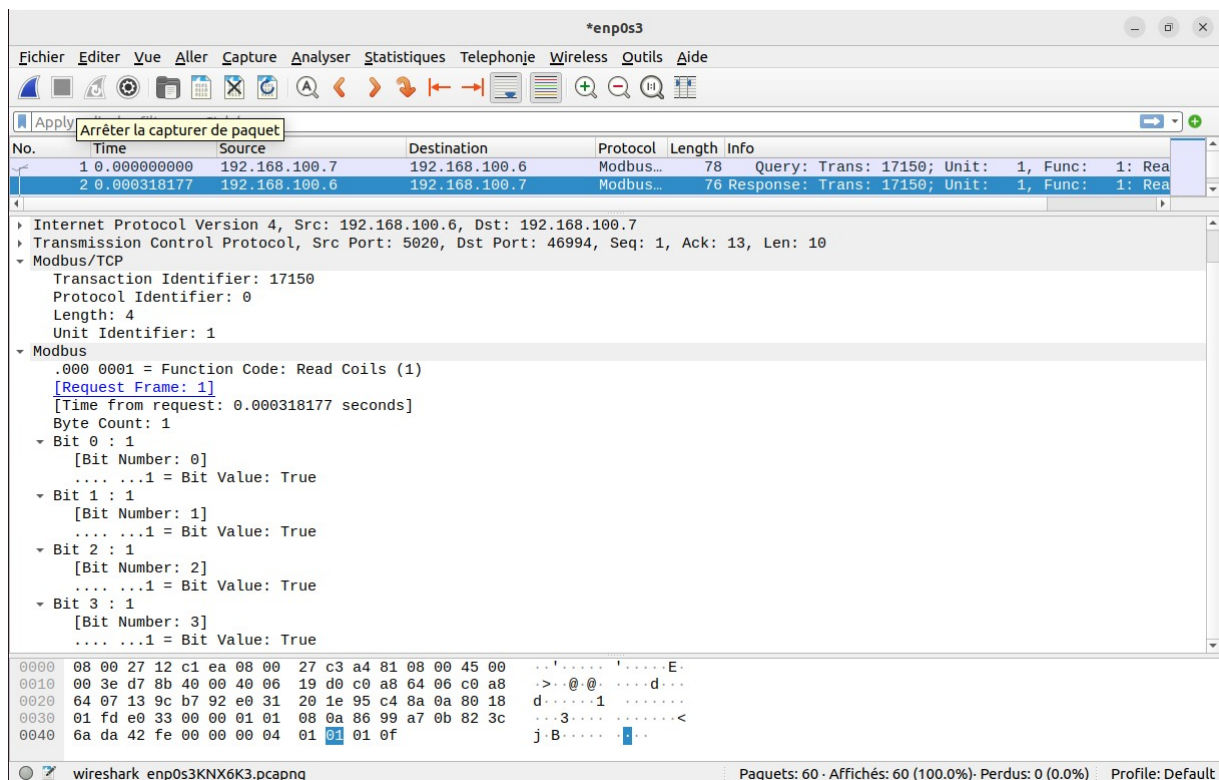
The screenshot shows the Wireshark interface with the following details:

- Packet List:** A list of captured packets. Packet 310 is highlighted, showing a TCP packet from 192.168.100.7 to 192.168.100.6, Seq=1057, Ack=1023, Len=0.
- Packet Details:**
 - Ethernet II, Src: PcsCompu_12:c1:ea (08:00:27:12:c1:ea), Dst: PcsCompu_c3:a4:81 (08:00:27:c3:a4:81)
 - Internet Protocol Version 4, Src: 192.168.100.7, Dst: 192.168.100.6
 - Transmission Control Protocol, Src Port: 49864, Dst Port: 5020, Seq: 1057, Ack: 1023, Len: 0
- Packet Bytes:** A hex dump of the packet data, showing the raw bytes of the Modbus TCP frame.

A savoir étant donné qu'on utilise pour ce TP le port 5020 pour Modbus et non le port 502, pour voir les détails d'une trame d'envoi d'information **Modbus TCP** il faut dans Wireshark aller dans:

1. Edit -> Preferences
2. Protocols -> Modbus
3. Dans « TCP Ports » ajouté 5020





Une trame Modbus TCP se compose de deux parties principales :

MBAP Header :

Champ	Taille	Description
Transaction Identifier	2 octets	Numéro unique pour identifier la requête
Protocol Identifier	2 octets	Toujours 00 00 pour Modbus TCP
Length	2 octets	Nombre d'octets qui suivent (Unit ID + PDU)
Unit Identifier	1 octet	Identifie l'esclave/automate destinataire

PDU (Protocol Data Unit, longueur variable) :

Champ	Taille	Description
Code de Fonction	1 octet	Action demandée (lecture/écriture)
Reference Number	2 octets	Adresse du registre ciblé
Données	N octets	Valeur lue ou écrite
Padding	0 ou 1 octet	Optionnel, pour aligner la trame si nécessaire

Total trame = MBAP header (7 octets) + PDU (variable)

Méthodologie pour décoder une trame

1. **Identifier le début de la trame Modbus TCP**
 - Ignore la partie **Ethernet (MAC)** et **IP/TCP**.
 - Repère la partie qui commence à être capturée sur le **port TCP 502** → c'est là que se trouve le **MBAP header**.
2. **Découper le MBAP header (7 octets)**
 - Transaction ID : premiers 2 octets
 - Protocol ID : octets suivants
 - Length : les 2 octets après Protocol ID
 - Unit ID : dernier octet du MBAP header
3. **Lire le PDU (après MBAP header)**
 - Code de Fonction → 1 octet
 - Reference Number → 2 octets (si la fonction concerne un registre spécifique)
 - Données → selon la longueur du PDU (indiquée par le champ Length)
 - Padding → vérifier si le nombre d'octets correspond au Length
4. **Identifier Requête ou Réponse**
 - Source IP = client → Requête
 - Source IP = automate/esclave → Réponse
5. **Vérifier la longueur**
 - Length (MBAP header) = nombre d'octets dans **Unit ID + PDU**.
 - Si plus d'octets → padding. Sinon → pas de padding.

Exemple 1 : Trame n°1 (Requête)

```
0000 | 5e 31 15 e9 09 47 d2 47   97 b6 c1 bf 08 00 45 00
0010 | 00 40 00 00 40 00 40 06   da b8 ac 12 03 dd ac 12
0020 | 03 fe c5 d2 01 f6 eb 93   49 1d da 97 98 2c 80 18
0030 | 08 02 d7 f3 00 00 01 01   08 0a 57 68 7f 94 f8 d9
0040 | f7 2f 00 63 00 00 00 06   01 05 00 00 ff 00
```

Trame (Modbus TCP uniquement) :

```
00 00 00 06 01 05 00 00 ff 00
```

Champ	Valeur	Explication
Transaction ID	00 00	ID unique de la requête
Protocol ID	00 00	Toujours 0
Length	00 06	6 octets suivent (Unit ID + PDU)
Unit ID	01	Automate cible
Code de Fonction	05	Write Single Coil (écrire une sortie)
Reference Number	00 00	Adresse du registre/bobine
Données	ff 00	Valeur à écrire
Padding	Aucun	La trame est complète

Exemple 2 : Trame n°2 (Réponse)

0000 | d2 47 97 b6 c1 bf 5e 31 15 e9 09 47 08 00 45 00
0010 | 00 3e 00 00 40 00 40 06 da ba ac 12 03 fe ac 12
0020 | 03 dd 01 f6 c5 d2 da 97 98 5c eb 93 49 59 80 18
0030 | 08 01 4e 65 00 00 01 01 08 0a f8 da 04 ca 57 68
0040 | 8c c1 6c b8 00 00 00 04 01 01 01 0f

Trame (Modbus TCP uniquement) :

00 00 00 04 01 01 01 0f

Champ	Valeur	Explication
Requête ou Réponse	Réponse	Envoyée par l'esclave
Transaction ID	00 00	Correspond à la requête initiale
Protocol ID	00 00	Toujours 0
Length	00 04	4 octets suivent (Unit ID + PDU)
Unit ID	01	Automate qui répond
Code de Fonction	01	Read Coils (lecture bobines)
Données	01 0f	Valeur lue dans les bobines
Padding	Aucun	La trame est complète

Astuces pratiques

1. Toujours compter les octets du MBAP header → 7 octets exacts.
2. Le champ Length → permet de savoir exactement combien d'octets il reste dans le PDU.
3. Requête vs Réponse → regarder la source IP/TCP et le code fonction :
 - Code fonction de lecture/écriture → Requête
 - Valeur retournée → Réponse
4. Padding → rare, apparaît seulement si le PDU n'a pas le bon alignement.
5. Hexadécimal → lisible : les outils comme Wireshark permettent de voir "valeur des bobines" directement, sinon lire les octets en hexadécimal.